

《操作系统》

lab5：用户程序

一. 实验要求

练习0：填写已有实验

本实验依赖实验2/3/4。请把你做的实验2/3/4的代码填入本实验中代码中有“LAB2”/“LAB3”/“LAB4”的注释相应部分。
注意：为了能够正确执行 lab5 的测试应用程序，可能需对已完成的实验2/3/4的代码进行进一步改进。

练习1：加载应用程序并执行（需要编码）

`do_execve` 函数调用 `load_icode`（位于 `kern/process/proc.c` 中）来加载并解析一个处于内存中的 ELF 执行文件格式的应用程序。你需要补充 `load_icode` 的第 6 步，建立相应的用户内存空间来放置应用程序的代码段、数据段等，且要设置好 `proc_struct` 结构中的成员变量 `trapframe` 中的内容，确保在执行此进程后，能够从应用程序设定的起始执行地址开始执行。需设置正确的 `trapframe` 内容。

请在实验报告中简要说明你的设计实现过程。

请简要描述这个用户态进程被 `ucore` 选择占用 CPU 执行（RUNNING 态）到具体执行应用程序第一条指令的整个经过。

练习2：父进程复制自己的内存空间给子进程（需要编码）

创建子进程的函数 `do_fork` 在执行中将拷贝当前进程（即父进程）的用户内存地址空间中的合法内容到新进程中（子进程），完成内存资源的复制。具体是通过 `copy_range` 函数（位于 `kern/mm/pmm.c` 中）实现的，请补充 `copy_range` 的实现，确保能够正确执行。

请在实验报告中简要说明你的设计实现过程。

练习3：阅读分析源代码，理解进程执行 `fork/exec/wait/exit` 的实现，以及系统调用的实现（不需要编码）

请在实验报告中简要说明你对 `fork / exec / wait / exit` 函数的分析，并回答如下问题：

- 请分析 `fork / exec / wait / exit` 的执行流程。重点关注哪些操作是在用户态完成，哪些是在内核态完成？内核态与用户态程序是如何交错执行的？内核态执行结果是如何返回给用户程序的？
- 请给出 `ucore` 中一个用户态进程的执行状态生命周期图（包执行状态，执行状态之间的变换关系，以及产生变换的事件或函数调用）。（字符方式画即可）
- 执行：`make grade`。如果所显示的应用程序检测都输出 `ok`，则基本正确。（使用的是 `qemu-4.1.1`）

扩展练习 Challenge

1.实现 Copy on Write (COW) 机制

给出实现源码、测试用例和设计报告（包括在 COW 情况下的各种状态转换（类似有限状态自动机）的说明）。

这个扩展练习涉及到本实验和上一个实验“虚拟内存管理”。在 ucore 操作系统中，当一个用户父进程创建自己的子进程时，父进程会把其申请的用户空间设置为只读，子进程可共享父进程占用的用户内存空间中的页面（这就是一个共享的资源）。当其中任何一个进程修改此用户内存空间中的某页面时，ucore 会通过 page fault 异常获知该操作，并完成拷贝内存页面，使得两个进程都有各自的内存页面。这样一个进程所做的修改不会被另外一个进程可见了。请在 ucore 中实现这样的 COW 机制。

由于 COW 实现比较复杂，容易引入 bug，请参考 <https://dirtycow.ninja/> 看看能否在 ucore 的 COW 实现中模拟这个错误和解决方案。需要有解释。

这是一个 big challenge。

2.说明该用户程序是何时被预先加载到内存中的？与我们常用操作系统的加载有何区别，原因是什么？

二. 实验流程

我们先将之前的Lab中的代码补充完整，随后开始练习实验代码补充：

练习1

```
//(6) setup trapframe for user environment
struct trapframe *tf = current->tf;
memset(tf, 0, sizeof(struct trapframe));
/* LAB5:EXERCISE1 YOUR CODE
 * should set tf_cs,tf_ds,tf_es,tf_ss,tf_esp,tf_eip,tf_eflags
 * NOTICE: If we set trapframe correctly, then the user level process can return to USER
MODE from kernel. So
 *      tf_cs should be USER_CS segment (see memlayout.h)
 *      tf_ds=tf_es=tf_ss should be USER_DS segment
 *      tf_esp should be the top addr of user stack (USTACKTOP)
 *      tf_eip should be the entry point of this binary program (elf->e_entry)
 *      tf_eflags should be set to enable computer to produce Interrupt
 */
// 设置中断帧，使得在内核态能返回到用户态
tf->gpr.sp = USTACKTOP;           // 用户栈顶
tf->epc = elf->e_entry;           // 用户程序入口点
tf->status = (read_csr(sstatus) & ~SSTATUS_SPP & ~SSTATUS_SPIE); // 设置为用户态，允许中断

ret = 0;
```

- 设计思路如下：

在完成 ELF 各 PT_LOAD 段的 VMA 建立、物理页分配与拷贝（含 BSS 清零）、以及用户栈区映射与栈页分配后，将新建 mm 绑定到 current 并切换 satp 使用该用户地址空间生效；随后初始化 current->tf，把 sp 设置为 USTACKTOP、把 epc 设置为 elf->e_entry，并配置 sstatus 使 SPP=0（确保 sret 回到用户态）且 SPIE=1（确保回到用户态后允许中断），从而保证该进程第一次被调度运行时能从 ELF 入口开始执行。

- 整体经过描述：

调度器 `schedule()` 在就绪队列中选中该进程后调用 `proc_run(next)`，`proc_run` 会把 `current` 切换为 `next`，并加载该进程的页表（你这里在 `load_icode` 里已经设置了 `current->pgdir`，运行时会通过切换 `satp`/刷新 TLB 让用户地址空间成为当前地址空间），然后通过 `switch_to` 做上下文切换，把 CPU 寄存器现场换成新进程的内核上下文。对“第一次运行的用户进程”而言，它的内核上下文通常被初始化为从一个内核入口（常见是 `forkret`）开始执行，`forkret` 再跳到 `trap_return` 一类的汇编例程，`trap_return` 会按 `current->tf` 恢复通用寄存器、设置用户栈指针、装载 `sepc=tf->epc` 与 `sstatus=tf->status`，最后执行 `sret`：CPU 特权级从 S-mode 切到 U-mode，PC 跳到 `elf->e_entry`，SP 变成 `USTACKTOP`，于是用户程序的第一条指令就在用户态被取指并执行。

练习2

`copy_range`的补充实现如下：

```
int ret = 0;

/* LAB5:EXERCISE2 YOUR CODE
 * (1) find src_kvaddr: the kernel virtual address of page
 * (2) find dst_kvaddr: the kernel virtual address of npage
 * (3) memory copy from src_kvaddr to dst_kvaddr, size is PGSIZE
 * (4) build the map of phy addr of npage with the linear addr start
 */

// (1) 获取源页面的内核虚拟地址
void *src_kvaddr = page2kva(page);

// (2) 获取目标页面的内核虚拟地址
void *dst_kvaddr = page2kva(npage);

// (3) 复制页面内容，大小为 PGSIZE
memcpy(dst_kvaddr, src_kvaddr, PGSIZE);

// (4) 建立目标进程的虚拟地址到新物理页面的映射
ret = page_insert(to, npage, start, perm);

assert(ret == 0);
```

- 设计思路如下：

`copy_range` 以页为单位遍历父进程在 `[start,end)` 的用户虚拟地址区间，查询父页表项，跳过不存在或无效的页；对每个有效用户页，分配新的物理页作为子进程对应页，通过 `page2kva` 获得父页与子页在内核中的可访问地址后执行 `memcpy` 完成整页内容复制，并调用 `page_insert` 把子页映射到子进程页表的相同虚拟地址，同时继承父页的用户态访问权限位，从而实现子进程用户地址空间内容的独立副本。

- 如何设计实现 Copy on Write (COW) 机制：

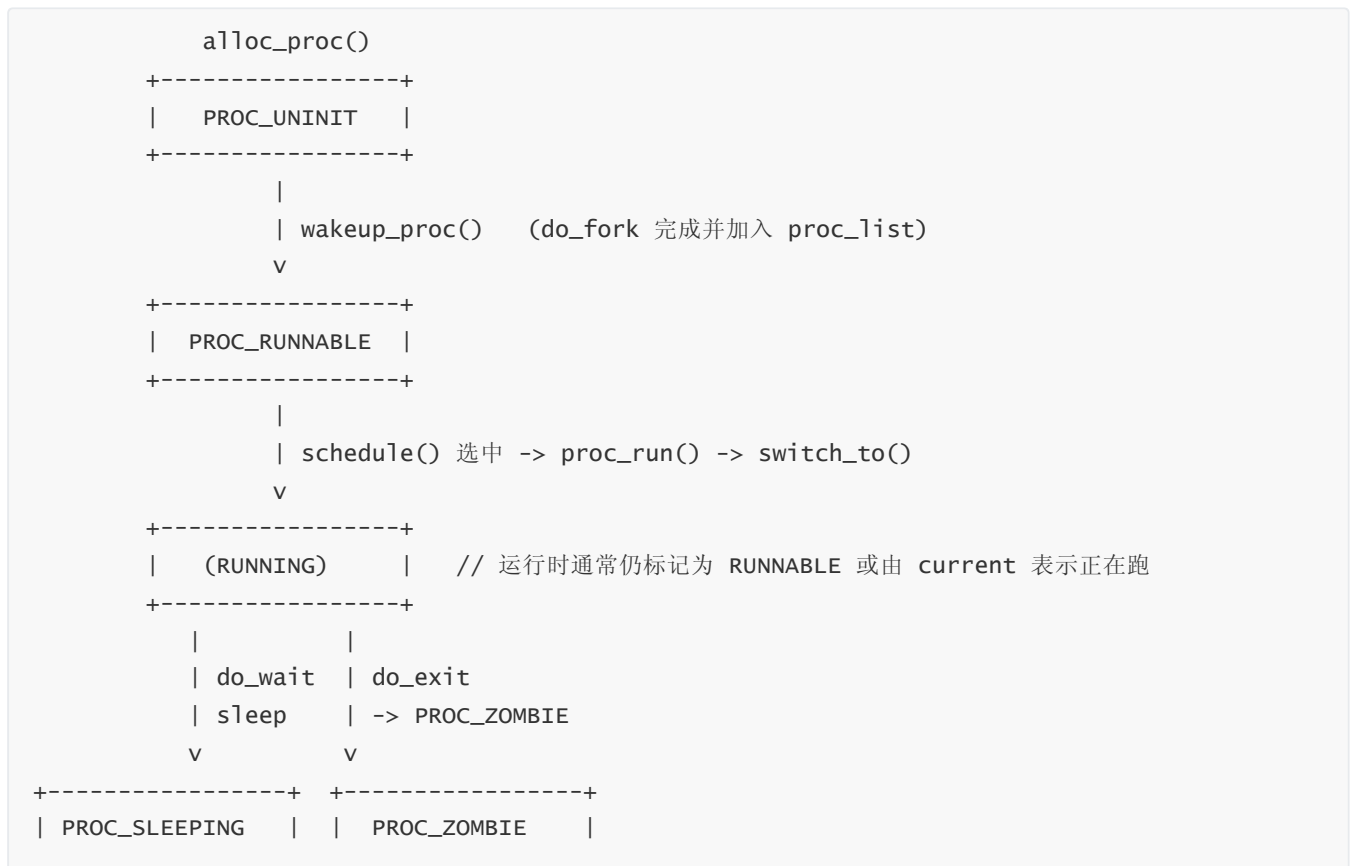
把 `fork` 时的“拷贝”推迟到“第一次写入”发生的那一刻：在 `do_fork/copy_range` 阶段不再为每个页分配新页并 `memcpy`，而是让父子进程暂时共享同一物理页，并把两边的 PTE 都改成“只读 + COW 标记”。实现上通常需要三块改动：其一，在 PTE 里找一个软件可用位（RISC-V PTE 有保留给软件使用的位，或你们框架提供了类似 `PTE_COW` 的宏）作为 COW 标志；其二，`fork` 时遍历父进程的可写用户页，把父页表项与子页表项都清掉写权限（去掉 `PTE_W`），并置上 `PTE_COW`，同时把该物理页的引用计数加一（`ucore` 里一般 `struct Page` 有 `ref` 计数，或者通过 `page_ref_inc/page_ref_dec` 维护）；其三，在缺页异常/页保护异常处理路径里（写触发的 `page fault`），检查 `fault` 的地址对应的 PTE 是否带 `PTE_COW`：如果是，说明这是对共享只读页的写入——此

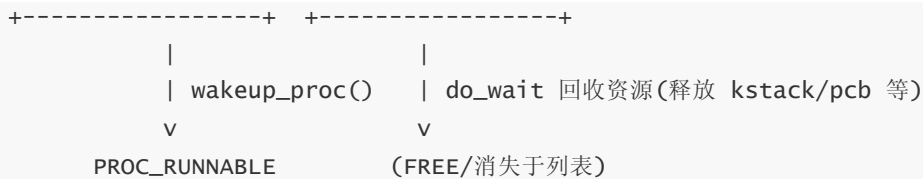
时若该物理页 `refcount>1`，就分配一个新页、把旧页内容复制到新页、把当前进程的 PTE 更新为新页并恢复可写（清 `PTE_COW`、加 `PTE_W`），同时把旧页 `refcount` 减一；若 `refcount==1`，说明当前进程已独占该页，不必拷贝，直接把 PTE 改回可写并清掉 `PTE_COW` 即可，最后刷新对应地址的 TLB（例如 `sfence.vma`）。这样就能保证语义上父子互不影响，同时在大多数页“只读不写”的场景下显著减少 fork 的复制开销。

练习3

用户程序在 U-mode 执行到 `fork()`/`exec()`/`wait()`/`exit()` 时，通过 `ecall` 陷入内核；陷入后进入 S-mode 的 trap handler，内核把当时的用户寄存器现场保存到 `current->tf` (trapframe)，再按系统调用号分发到 `sys_fork/sys_exec/sys_wait/sys_exit`，真正的核心工作分别在 `do_fork/do_execve/do_wait/do_exit` 中完成。`fork` 在内核里分配子进程 PCB、内核栈，复制/共享父进程的用户地址空间（`copy_mm` 及其内部的 `copy_range` 或 COW），并用 `copy_thread` 复制父进程 trapframe 到子进程内核栈顶，同时把子进程 trapframe 的 `a0` 置 0、把其内核上下文 `ra` 设为 `forkret`，再把子进程置为 `PROC_RUNNABLE`；父进程这次系统调用在内核直接返回子 pid。`exec` 在内核里先切回内核页表并释放旧 mm/页表/VMA，再 `load_icode` 解析 ELF、建立新用户空间映射与用户栈，并重置 `current->tf` 的关键字段（如 `sp=USTACKTOP`、`epc=elf->e_entry`、状态位返回用户态），所以回到用户态时从新程序入口开始执行。`wait` 在内核检查目标子进程是否退出：没退出就把父进程置为 `PROC_SLEEPING` 并 `schedule()` 让出 CPU；子进程 `exit` 时在内核把自己置为 `PROC_ZOMBIE`、记录退出码并唤醒父进程，父进程之后再被调度回来继续执行 `do_wait` 完成回收并返回。`exit` 的核心同样在内核：释放/减少引用资源、设置 `PROC_ZOMBIE`、唤醒父进程并触发调度，正常情况下不会再返回到用户态。

内核态与用户态的交错方式是“用户态运行一段→遇到系统调用/异常/中断陷入内核→内核短暂运行完成管理操作→通过 trap return 恢复现场回到用户态继续”，其中 `wait` 是会把“返回用户态”推迟到事件发生之后的一类：它在内核里把进程睡眠并切走，等被唤醒后再继续把结果返回。内核态执行结果返回给用户程序的机制是：`do_*` 的返回值一路返回到系统调用入口，由入口把返回值写入约定的返回寄存器（RISC-V 通常是 `a0`，对 `fork` 来说父进程是子 pid；子进程则由 `copy_thread` 预先将其 trapframe 的 `a0` 改为 0），然后 trap return 根据 `current->tf` 恢复寄存器并执行 `sret` 回到用户态；用户态看到的就是一次普通函数返回（`exec` 则因为重置了 `epc/sp`，返回后直接从新入口开始跑；`exit` 正常不再返回）。





lab2分支任务

目标: 观察用户态→内核态→用户态的完整系统调用过程 (U mode → S mode → U mode)

核心流程

1. 加载用户程序符号表

```
# 终端3 (调试ucore的gdb)
make gdb
(gdb) add-symbol-file obj/__user_exit.out
y
```

原因: makefile只加载了内核符号, 用户程序需手动加载

2. 观察 ecall (用户态→内核态)

步骤A: 在 syscall 打断点

```
(gdb) break user/libs/syscall.c:18
(gdb) continue
Breakpoint 1, syscall (...) at user/libs/syscall.c:19
```

步骤B: 找到 ecall 指令

```
(gdb) disassemble
...
0x800104 <syscall+44>:  ecall    ← 目标指令
...

(gdb) break *0x800104          # 在ecall前打断点
(gdb) continue
(gdb) stepi                   # 执行到ecall指令
```

步骤C: 为QEMU设置断点

```
# 终端2 (调试QEMU的gdb)
# 按 Ctrl+C 中断
(gdb) break riscv_cpu_do_interrupt    # QEMU处理中断的入口
(gdb) continue

# 回到终端3
(gdb) stepi    # 执行ecall指令
# 终端2会触发断点，显示QEMU如何处理ecall
```

3. 观察系统调用处理（内核态）

```
# 终端3: ecall后会进入 __alltraps → trap → syscall
(gdb) break syscall    # 内核的系统调用处理函数
(gdb) continue
# 观察系统调用的处理过程
```

4. 观察 sret（内核态→用户态）

步骤A: 找到返回点

```
(gdb) break __trapret    # 中断返回入口
(gdb) continue

(gdb) disassemble
...
0xfffffffffc0... <__trapret+XX>:  sret    ← 目标指令
...

(gdb) break *0xfffffffffc0...    # 在sret前打断点
(gdb) continue
```

步骤B: 为QEMU设置断点

```
# 终端2
# 按 Ctrl+C 中断
(gdb) break helper_sret    # QEMU处理sret的函数
(gdb) continue

# 回到终端3
(gdb) stepi    # 执行sret指令
# 终端2触发断点，显示QEMU如何处理sret
```

5. 验证返回用户态

```
# 终端3: sret后应该返回到用户程序
(gdb) info registers pc
pc = 0x800108      # 回到用户态地址空间

(gdb) info registers sstatus
# 查看 SPP 位, 应该是 0 (用户态)
```

关键观察点

ecall 处理 (QEMU源码)

```
riscv_cpu_do_interrupt()      [cpu_helper.c]
↓
设置 sepc = 当前PC            (保存返回地址)
设置 scause = ECALL_FROM_U_MODE (保存异常原因)
设置 sstatus.SPP = U          (保存之前的特权级)
设置 PC = stvec                (跳转到中断处理入口)
切换到 S 模式
```

sret 处理 (QEMU源码)

```
helper_sret()                  [op_helper.c]
↓
恢复 PC = sepc                (返回用户态地址)
恢复特权级 = sstatus.SPP      (恢复为U模式)
清除 sstatus.SPP              (清理状态位)
```

1. 关键调用路径、关键分支，以及用调试“演示”一次虚拟地址到物理地址的翻译

在 QEMU-4.1.1 (TCG) 里，一条访存指令最终会走到“先查 TLB、TLB miss 再做软件页表遍历、最后把翻译结果灌回 TLB 并完成访存”的主链路：**TCG 生成的访存 helper (如 `cpu_ld/cpu_st` 家族)** → **`accel/tcg/cputlb.c` 的 TLB 快路径查找** → **miss 时进入 `riscv_cpu_tlb_fill()`** → **`get_physical_address()` (RISC-V MMU 翻译核心)** → **`tlb_set_page_with_attrs()` 写回 QEMU 的 TLB**，之后同一页内的访存会命中快路径。你之前打断点的点位正好覆盖了这条链 (`riscv_cpu_tlb_fill / get_physical_address if env->satp != 0 / tlb_set_page_with_attrs`)。

关键分支通常集中在：

- (a) **satp 是否为 0** (`satp==0` 走 Bare/无翻译, `VA≈PA`; `satp!=0` 才进入 Sv32/Sv39/Sv48 页表遍历)
- (b) **访问类型** (取指/读/写走不同权限检查)
- (c) **当前特权级与 mstatus 位** (例如 SUM/MXR 影响 U/S 访问语义)
- (d) **PTE 位检查** (`V/R/W/X/U/A/D` 的合法性与缺页/权限异常分支)
- (e) **超级页/叶子 PTE 的判定** (在多级遍历中遇到叶子就停止并计算物理页号)

- **(f) PMP/物理内存属性**（即便页表放行，仍可能被 PMP/设备属性拒绝）

调试“演示”时可以选定某条触发访存的指令，命中 `riscv_cpu_tlb_fill` 后在 GDB 里观察：

- `addr`（虚拟地址）
- `access_type`（R/W/X）
- `env->satp`（模式与根页表 PPN）
- `get_physical_address` 的输出（PA、权限/属性）

再在 `tlb_set_page_with_attrs` 处确认 QEMU 把“VA 页→PA 页”的映射按页粒度写入了 TLB。随后同页再次访问会直接走 `cputlb.c` 的命中路径而不再进 `get_physical_address`。

2. 单步调试页表翻译：关键操作流程是什么

把 `get_physical_address()` 当作“RISC-V 软件页表遍历的实现体”来单步最直观：它会先从 `satp` 解析出 **地址转换模式 (Sv39/Sv48/...)** 和 **根页表物理基址**，根据 VA 切分出各级 VPN（例如 Sv39 是 3 级 VPN + page offset），然后进入一个“按级循环”。

循环中的关键步骤：

1. **用当前级 VPN 索引读取 PTE**（从内存取 PTE 这一步在 QEMU 里也是一次受控的物理访存）
2. **检查 V 与 R/W/X 组合是否合法**
3. **判定 PTE：**
 - 若不是叶子，则把 PTE 指向的下一级页表作为新基址继续下探
 - 若是叶子，则计算最终 PA（拼出 PPN 与页内偏移，同时处理超级页对低位 PPN 的特殊拼接规则）
4. **做权限/特权检查**（U 位、SUM/MXR、取指/读/写差异）
5. **检查/更新 A/D 位语义**（QEMU 往往以“触发异常或软件置位”的方式模拟，取决于实现策略）
6. **最终返回**“翻译成功/异常原因+异常信息（stval/scause 语义）”

单步时最值得解释的“关键动作”就是：每一级的 PTE 读取与判定（是否叶子/是否继续）、以及成功时把“页粒度翻译结果+权限属性”交还给上层，让上层在 `tlb_set_page_with_attrs()` 里把它缓存起来，从而把“昂贵的多级遍历”摊薄到“每页第一次访问一次”。

3. 能否在 QEMU-4.1.1 找到“模拟 CPU 查 TLB”的 C 代码，并用调试说明细节

能，QEMU 的 **TLB 快路径**在 `accel/tcg/cputlb.c`（以及相关头文件如 `include/exec/cpudefs.h`/`include/exec/cputlb.h` 一带）里实现。

访存 helper 先用 **虚拟地址计算 TLB 索引**（通常是按页号哈希/取低位），取出对应的 `CPUTLBEntry`（按 ASID/特权/访问类型分组），做几步非常“CPU味儿”的快速判断：

1. **比较 tag**（页号/地址空间标识）是否匹配
2. **检查允许的访问类型**（读/写/取指）是否在条目权限里
3. **通过条目里缓存的 addend/host 指针**把 guest 地址快速映射到 host RAM 指针或触发慢路径

任何一步失败都会走到慢路径 `tlb_fill`，最后落到你打断点命中的 `riscv_cpu_tlb_fill()` 与 `tlb_set_page_with_attrs()`。

调试时如果你在 `cputlb.c` 的“lookup/fast path”位置再补一个断点（或在 `cpu_ld*/cpu_st*` 附近下断点），你会看到：

- **第一次访问**：先 lookup → miss → fill → set_page
- **第二次访问**：lookup → hit → 直接算出 host 地址完成访存

这就是“QEMU 里模拟 TLB 的价值”以及它和页表遍历函数之间的接口边界。

4. QEMU 的“TLB”与真实 CPU 的 TLB：逻辑上的区别是什么

逻辑上两者都在做“缓存 VA→PA 的页粒度翻译以避免重复页表遍历”，但 QEMU 的 TLB 是软件数据结构服务于仿真执行引擎（TCG），因此它的行为更“工程化”。

QEMU TLB 的特点：

- 往往把 **权限/内存属性**（MMIO、只读、可执行、脏/访问语义的处理策略等）与翻译结果一起缓存
- 为了加速，会直接缓存到“可用于 host 访存”的形式（例如 host 指针偏移/addend），从而把后续访存变成极少量的 C 代码

真实 CPU TLB 的特点：

- 是 **硬件并行查找、与流水线/异常精确性强耦合**
- 还与 I-TLB/D-TLB 分离、分层 TLB、硬件 page-walk cache、ASID/PCID、一致性/shootdown 协议 等微结构紧密绑定

核心区别：

- 真实 TLB 的“命中/未命中”是 **硬件时序事件**
- QEMU 的“命中/未命中”是 **软件分支选择**
- 真实 TLB 的条目格式与更新受 **ISA + 微结构** 约束
- QEMU 的条目格式与更新策略主要受 **如何最快把 guest 内存访问变成 host 内存访问** 驱动（并且会显式区分 RAM 与 MMIO、处理跨页、处理异常回填等）

这也是为什么你在 QEMU 里能清晰地用断点把“lookup→fill→set_page”这条链完整抓出来，而在真实 CPU 上它更多是不可见的硬件行为。

lab5分支任务

1. ecall / sret 在 QEMU 中是怎么被处理的（结合这次双 gdb 调试）

这次调试我用三个终端 + 两个 gdb：

- 终端 1：make debug 启动带调试信息的 QEMU，并暂停在 reset。
- 终端 2：make gdb 用 `riscv64-unknown-elf-gdb` 连接到 QEMU，调试运行在里面的 ucore + 用户态程序。
- 终端 3：在 QEMU 源码目录下 `gdb ./riscv64-softmmu/qemu-system-riscv64`，attach 到正在跑的 QEMU 进程，调 QEMU 自己的 C 源码。

(1) 从用户态 syscall() 到 ecall 被 QEMU 接住

1. 在终端 2 里，我先用

```
gdb
add-symbol-file obj/__user_exit.out
break user/libs/syscall.c: 18
c
```

让 gdb 在用户态的 syscall() 封装处停下来。

2. 停在 syscall() 后，用

```
gdb
x/7i $pc
```

可以看到那一段汇编里有一条 ecall，然后用 stepi 一步步走，停到

```
=> ... <syscall+...>: ecall
```

也就是准备执行 ecall 但还没执行。

3. 这时切到终端 3，attach 上 QEMU，`handle SIGPIPE nostop noprint`，在 QEMU 源码里用 `grep "ECALL"` 找到 RISC-V 的特权指令翻译/处理代码，比如：

- `target/riscv/insn_trans/trans_privileged.inc.c` 里有 `gen_ecall(...)` / `generate_exception(...)`
- `target/riscv/cpu_helper.c` / `op_helper.c` 里有 `riscv_cpu_do_interrupt(...)` 等异常处理入口

然后在异常处理入口打断点

```
gdb
break riscv_cpu_do_interrupt
continue
```

4. 回到终端 2，对那条 ecall 再来一次 `stepi`，这一步真正执行了 guest 的 ecall。

结果：终端 2 卡住不动，终端 3 的 gdb 在 `riscv_cpu_do_interrupt`（或相关 helper）断点处停下。

5. 在 QEMU gdb 里，通过 `bt` / `info args` / `info locals` 可以看到大致流程：

- QEMU 已经在前面的 TCG 翻译阶段把 RISC-V 的 ecall 翻译成了一个抛出异常的 helper 调用（见 `gen_ecall` → `generate_exception`）。
- 真正运行时，进入 `riscv_cpu_do_interrupt` 这一类函数，根据异常原因（U 模式 ecall）、当前 `env->pc` 等：
 - 设置模拟的 CSR：`scause = ECALL_FROM_U`、`sepc = 当前 PC`、`stval` 等。
 - 根据 `stvec` 计算新 PC，跳到内核 trap 入口（`__alltraps`）。
 - 更新 `priv`（特权级）从 U → S。

也就是说，从用户态 ecall 到 S 态 trap 入口，硬件本来要做的那套“设置 CSR + 切换 PC + 切换特权级”，在 QEMU 里由 C 函数完成，我们能在 gdb 里清楚地看到。

6. 继续 `c` 之后，QEMU 回到"执行 guest 指令"，此时 ucore 已经进入到 S 态的 trap 汇编 `__alltraps`，保存寄存器，构造 trapframe，再跳到 C 函数 `trap()` / `exception_handler()`，最后转发到 `syscall()`，走完内核里的系统调用处理逻辑。

(2) 从 S 态返回 U 态：__trapret + sret 在 QEMU 里的处理

1. 内核完成系统调用后，在终端 2 的 riscv-gdb 里，我在 trap 返回的汇编位置打断点：

```
gdb
break __trapret
continue
```

2. 触发一次系统调用后，gdb 在 `__trapret` 处停下。查看附近汇编：

```
gdb
x/10i $pc
```

可以看到一长串 LOAD 恢复寄存器，最后有一条 `sret`。
用 `stepi` 走到：

```
=> ...: sret
```

同样是还没真正执行 `sret`。

3. 此时再次切到终端 3（QEMU gdb），我们提前用 `grep "helper_sret"` 找到了 `sret` 的 helper：

- `target/riscv/op_helper.c: helper_sret(...)`
- 对应的翻译宏在 `target/riscv/insn_trans/trans_privileged.inc.c: gen_helper_sret(...)`

在 QEMU 里打断点：

```
gdb
break helper_sret
continue
```

4. 回到终端 2，对 `sret` 执行 `stepi`，真正执行 guest 的 `sret` 指令。

结果：终端 2 又卡住，终端 3 在 `helper_sret` 断点处停下（你已经看到了 `helper_sret(env, cpu_pc_deb)` 那一行）。

5. 在 `helper_sret` 里查看代码和局部变量，可以看到典型的 RISC-V `sret` 处理逻辑：

- 检查当前特权级 `env->priv` 是否允许执行 `sret`（必须来自 S/M）。
- 从 `sstatus.SPP`、`SIE/SPIE` 等位推导出返回后的特权级和中断使能状态。
- 从 `sepc` 恢复返回地址，设置 `env->pc = sepc`。
- 把 `env->priv` 改成返回后的模式（这里是回到 U）。
- 更新 `sstatus` 中 `SPP/SIE/SPIE` 位，模拟硬件状态变化。

也就是说，从 S 态 trap 返回 U 态那一瞬间，QEMU 通过 `helper_sret` 这个 C 函数完整模拟了硬件对 CSR 和特权级的更新，我们可以在断点处一行行看清楚。

6. `continue` 之后，控制权回到 `guest: ucore` 已经回到了用户态进程，继续在 `sepc` 指向的那条用户指令之后执行。

2. TCG 指令翻译在 `ecall` / `sret` 中起了什么作用？和另一个双 `gdb` 实验有什么关系？

QEMU 并不是"逐条解释执行 RISC-V 指令"，而是用它自己的 **TCG (Tiny Code Generator)** 做一个翻译层：

1. 每当遇到一段 `guest` 代码（RISC-V 指令流），QEMU 先调用翻译器（`target/riscv/translate.c` 以及各种 `insn_trans/*.inc.c`），把这些指令翻译成一串 TCG 中间表示（类似中间代码/微指令）。
2. TCG 会把这些中间代码编译成 `host`（x86-64）指令，再执行。执行的时候，就不再逐条重新解码 RISC-V 指令了，而是直接跑这些已翻译的块（TB, translation block）。

对于 `ecall/sret` 这种特殊指令，翻译阶段做的是：

- 在 `trans_privileged.inc.c` 里，`ecall` 被翻译为调用一个"抛异常的 helper"，比如：

```
gen_ecall(ctx); // 里面大致是 generate_exception(ctx, RISCV_EXCP_U_ECALL);
```

这会在 TCG IR 中插入"调用 `helper_raise_exception` / 触发异常"的操作。

- 同理，`sret` 在翻译时会变成一次对 `helper_sret(env, pc)` 的调用：

```
gen_helper_sret(cpu_pc, cpu_env, cpu_pc);
```

执行到这里，就会跳进我们打断点的 `helper_sret()` C 函数。

所以我们在 QEMU `gdb` 里看到的 `riscv_cpu_do_interrupt`、`helper_sret`，其实都是 **TCG 翻译出来的** "helper 函数"最终被执行的样子，这一层就是"指令翻译"。

和另一个"双重 `gdb` 调试页表查询过程"的实验的关系：

- 在"页表查询"的双 `gdb` 实验里，我们关注的是：访存指令（如 `lw`, `sw`）被 TCG 翻译后，如何在 QEMU 中触发 TLB 查找、TLB miss、页表遍历等流程。那一次在 QEMU 里应该是打在类似 `tlb_fill` / `riscv_cpu_do_transaction` / 地址翻译函数上。
- 这次"系统调用 + 返回"的实验，关注点从"地址翻译"换成了"特权级切换 + 异常/中断处理"，但底层机制是相同的：
 - `guest` 指令先被 TCG 翻译：
 - 访存指令 → 生成 `load/store` TCG op → 调到 `tlb/页表 helper`。
 - `ecall/sret` → 生成异常 `helper` / `helper_sret` 调用。
- 真正执行时，都是已经翻译好的 TCG 块在跑，遇到这些 `helper` 调用就跳到对应的 C 函数（地址翻译 / 异常处理 / 特权切换）。